



Operadores e Condicionais

Aula 03 · Bloco 1 — Fundamentos

Comparar do jeito certo e tomar decisões

Nesta aula

1. Operadores aritméticos — e o poder do `%`
2. `==` × `===` — a pegadinha mais famosa do JS
3. Operadores lógicos e `if` / `else`
4. **Truthy e falsy**
5. Ternário e `switch`

Aritméticos

```
console.log(10 + 3);    // 13
console.log(10 / 3);    // 3.3333...
console.log(10 % 3);    // 1 ← o RESTO da divisão
console.log(10 ** 3);   // 1000 (potência)
```

💡 O `%` parece inútil até você perceber que ele responde: "é par?" → `n % 2 === 0` . "é múltiplo de 5?" → `n % 5 === 0` . Aparece o tempo todo.

Atribuição composta

```
let pontos = 10;  
pontos += 5;    // pontos = pontos + 5 → 15  
pontos -= 3;    // 12  
pontos *= 2;    // 24  
pontos++;       // 25 – incrementa 1  
pontos--;       // 24 – decrementa 1
```

O `i++` vai aparecer em **todo** laço `for` que você escrever daqui em diante.

`==` e `===`: o JS tem dois "igual"

```
// == (frouxo): converte os tipos antes de comparar 😊  
console.log(5 == "5");    // true – converteu a string!  
console.log(0 == false);  // true (?!)  
console.log("" == false); // true (?!?!)  
  
// === (estrito): compara valor E tipo ✅  
console.log(5 === "5");   // false  
console.log(0 === false); // false
```

A regra do curso — e do mercado

Use **sempre** `===` e `!==`.

Esqueça que o `==` existe.

As conversões automáticas do `==` são fonte clássica de bug — e o pior tipo: aquele que não dá erro, só dá resposta errada.

Operadores lógicos

```
const idade = 20;  
const temIngresso = true;  
  
console.log(idade >= 18 && temIngresso); // E: só se AMBOS true  
console.log(idade >= 18 || temIngresso); // OU: se PELO MENOS UM  
console.log(!temIngresso); // NÃO: inverte → false
```

Decidindo: `if` / `else if` / `else`

```
if (media >= 7) {  
  console.log("Aprovado! 🎉");  
} else if (media >= 4) {  
  console.log("Recuperação. 📖");  
} else {  
  console.log("Reprovado. 😞");  
}
```

💡 A ordem importa: o JS testa de cima para baixo e **para na primeira condição verdadeira**.

Os seis valores *falsy*

Dentro de um `if`, o JS aceita **qualquer valor** — não só booleano.

false 0 "" null undefined NaN

Todo o resto é *truthy*. Decore estes seis e você sabe o resto por eliminação.

Truthy e falsy na prática

```
const nome = "";  
  
if (nome) {  
  console.log(`Olá, ${nome}`);  
} else {  
  console.log("Você não informou o nome!"); // ← cai aqui  
}
```

String vazia é falsy. Por isso `if (nome)` já serve como "o usuário preencheu?".

⚠ Cuidado: a string `"0"` **não** é vazia — logo, é **truthy**.

Ternário — o `if` de uma linha

```
const idade = 17;  
const status = idade >= 18 ? "maior de idade" : "menor de idade";  
//           -condição-   -se true-----   -se false-----
```

Use para escolher **um valor**, de forma simples.

Se precisar de mais de uma ação, prefira o `if` — ternário aninhado vira charada.

switch — muitos casos exatos

```
switch (diaDaSemana) {  
  case 1:  
    console.log("Domingo");  
    break;           // sem o break, "vaza" para o próximo caso!  
  case 2:  
    console.log("Segunda");  
    break;  
  default:           // o "else" do switch  
    console.log("Dia inválido");  
}
```

Compara com `===`. Esquecer o `break` é bug clássico — e o `switch` do Java é praticamente idêntico a este.

Exercícios da aula

Na pasta `aula-03/` :

1. `ex01.js` — par ou ímpar; positivo, negativo ou zero;
2. `ex02.js` — calculadora de IMC com as quatro faixas;
3. `ex03.js` — média de três notas + a regra de frequência;
4. `ex04.js` — `switch` do número ao nome do mês. E o que acontece sem os `break` ?
5. **Desafio** 🌶️ `ex06.js` — ano bissexto: teste com 2024 ✅, 1900 ❌ e 2000 ✅.

Próxima aula

Aula 04 — Laços e Funções

Repetir sem repetir código —
e o conceito mais importante do curso.